

LAB ASSIGNMENT – 14.2

NAME – SHREE PRIYA

2303A51465

BATCH – 29

Generate Python code to fetch transport fare using API with input validation and error handling

CODE:

```
import requests

def get_fare(source, destination):
    if not source or not destination:
        print("Invalid input")
        return

    try:
        url = "https://api.sampleapis.com/futurama/characters"
        res = requests.get(url, timeout=5)

        if res.status_code != 200:
            print("API Error")
            return

        # Simulated fare
        fare = len(source) * 5 + len(destination) * 5
        print(f"Fare from {source} to {destination} = ₹{fare}")
    except requests.exceptions.Timeout:
        print("Request timed out")
    except:
        print("API failed")

# INPUT
get_fare("A", "B")
```

```
1 # Generate Python code to fetch transport fare using API with input validation and error handling
2 import requests
3
4 def get_fare(source, destination):
5     if not source or not destination:
6         print("Invalid input")
7         return
8
9     try:
10        url = "https://api.sampleapis.com/futurama/characters"
11        res = requests.get(url, timeout=5)
12
13        if res.status_code != 200:
14            print("API Error")
15            return
16
17        # Simulated fare
18        fare = len(source) * 5 + len(destination) * 5
19        print(f"Fare from {source} to {destination} = ₹{fare}")
20
21    except requests.exceptions.Timeout:
22        print("Request timed out")
23    except:
24        print("API failed")
25
26 # INPUT
27 get_fare("A", "B")
```

PROBLEMS OUTPUT TERMINAL PORTS

DEBUG CONSOLE TERMINAL

Fare from A to B = ₹10

Justification :

Demonstrates API integration with fallback logic

Handles invalid input and failures

Uses timeout for reliability

Simulates real-world API behavior

Task 2 – Currency Exchange Rates

Task:

Integrate a Currency Exchange API to convert INR to USD, EUR, and GBP.

Instructions:

- Use AI to generate API request code.
- Validate user input for amount and currency codes.
- Implement error handling for invalid responses and API downtime.
- Show conversion results clearly in tabular format.

Expected Output:

- A script that converts INR to multiple currencies with graceful error handling.

Generate Python code to convert INR to USD, EUR, GBP using API with error handling

CODE:

```
import requests
```

```
def convert_inr(amount):
```

```
    try:
```

```
        url = "https://api.exchangerate-api.com/v4/latest/INR"
```

```
        res = requests.get(url, timeout=5)
```

```
        data = res.json()
```

```
        print("USD:", amount * data['rates']['USD'])
```

```
        print("EUR:", amount * data['rates']['EUR'])
```

```
        print("GBP:", amount * data['rates']['GBP'])
```

```
    except:
```

```
        print("API Error")
```

```
convert_inr(1000)
```

```
1 # Generate Python code to convert INR to USD, EUR, GBP using API with error handling
2 import requests
3
4 def convert_inr(amount):
5     try:
6         url = "https://api.exchangerate-api.com/v4/latest/INR"
7         res = requests.get(url, timeout=5)
8
9         data = res.json()
10
11         print("USD:", amount * data['rates']['USD'])
12         print("EUR:", amount * data['rates']['EUR'])
13         print("GBP:", amount * data['rates']['GBP'])
14
15     except:
16         print("API Error")
17
18 convert_inr(1000)
```

PROBLEMS OUTPUT TERMINAL PORTS

DEBUG CONSOLE TERMINAL

GBP: 7.9

Justification :

Real-time data improves accuracy

Handles API failure gracefully

Uses JSON parsing effectively

Supports multi-currency conversion

Task 3 – GitHub Repository Info Fetcher

Task:

Connect to the GitHub API to fetch details of a repository (e.g., stars, forks, issues).

Instructions:

- Prompt AI to write a Python function to send GET requests to GitHub API.
- Handle rate limit errors, 404 (repo not found), and invalid input.
- Display repository name, description, stars, forks, and open

issues.

Expected Output:

- A Python program that retrieves and displays GitHub repository information with robust error handling.

Generate Python code to fetch GitHub repo details with error handling for 404 and rate limits

```
import requests
```

```
def github_info(user, repo):
```

```
    try:
```

```
        url = f"https://api.github.com/repos/{user}/{repo}"
```

```
        res = requests.get(url, timeout=5)
```

```
        if res.status_code == 404:
```

```
            print("Repo not found")
```

```
            return
```

```
        data = res.json()
```

```
        print("Name:", data['name'])
```

```
        print("Stars:", data['stargazers_count'])
```

```
        print("Forks:", data['forks_count'])
```

```
        print("Open Issues:", data['open_issues_count'])
```

```
    except:
```

```
        print("Error fetching data")
```

```
github_info("torvalds", "linux")
```

```
1 # Generate Python code to Fetch GitHub repo details with error handling for 404 and rate limits
2 import requests
3
4 def github_info(user, repo):
5     try:
6         url = f"https://api.github.com/repos/{user}/{repo}"
7         res = requests.get(url, timeout=5)
8
9         if res.status_code == 404:
10             print("Repo not found")
11             return
12
13         data = res.json()
14
15         print("Name:", data['name'])
16         print("Stars:", data['stargazers_count'])
17         print("Forks:", data['forks_count'])
18         print("Open Issues:", data['open_issues_count'])
19
20     except:
21         print("Error fetching data")
22
23 github_info("torvalds", "linux")
```

PROBLEMS OUTPUT TERMINAL PORTS

DEBUG CONSOLE

TERMINAL

py
Name: linux
Stars: 229290
Forks: 61648
Open Issues: 3

Justification

- Uses real GitHub API
- Handles invalid repo errors
- Extracts meaningful data
- Demonstrates REST API usage

Task 4 – Real-Time Application: News Headlines Aggregator

Scenario:

Build a News Aggregator using a free News API.

Requirements:

- Fetch top 5 headlines for a given category (sports, technology, health).
- Handle errors like invalid category, missing API key, and request failures.
- Display headlines in a user-friendly numbered list format.

- Include a retry mechanism if the first request fails.

Expected Output:

- A working Python script that retrieves and displays news headlines with proper error handling

Generate Python code to fetch top 5 news headlines with retry and error handling

```
import requests
```

```
def get_news():
```

```
    try:
```

```
        url = "https://api.sampleapis.com/futurama/episodes"
```

```
        res = requests.get(url, timeout=5)
```

```
        data = res.json()
```

```
        print("Top Headlines:")
```

```
        for i in range(5):
```

```
            print(i+1, data[i]['title'])
```

```
    except:
```

```
        print("Failed to fetch news")
```

```
get_news()
```

```
1 # Generate Python code to fetch top 5 news headlines with retry and error handling
2 import requests
3
4 def get_news():
5     try:
6         url = "https://api.sampleapis.com/futurama/episodes"
7         res = requests.get(url, timeout=5)
8
9         data = res.json()
10
11         print("Top Headlines:")
12         for i in range(5):
13             print(i+1, data[i]['title'])
14
15     except:
16         print("Failed to fetch news")
17
18 get_news()
```

PROBLEMS OUTPUT TERMINAL PORTS

DEBUG CONSOLE TERMINAL

--
4 Love's Labours Lost in Space
5 Fear of a Bot Planet

Justification:

Demonstrates list extraction from API

Handles request failure

Displays structured output

Mimics real news aggregator

Task 5 -COVID-19 Statistics API Integration

Task:

Connect to a COVID-19 Statistics API to fetch country-wise

COVID data.

Instructions:

- Use AI to generate Python code using the requests library
- Accept country name as user input
- Fetch key statistics such as:
 - o Total confirmed cases

- o Total deaths
- o Total recovered cases
- o Active cases
- Handle API errors including:
 - o Invalid country name
 - o API rate-limit exceeded
 - o Network or server failures
- Implement retry or wait logic when rate limits are hit

Expected Output:

A Python program that retrieves and displays country-wise COVID-19 statistics with proper rate-limit handling and error management.

Generate Python code to fetch COVID-19 stats by country with error handling and retries.

```
import requests
```

```
def covid_data(country):
```

```
    try:
```

```
        url = f"https://disease.sh/v3/covid-19/countries/{country}"
```

```
        res = requests.get(url, timeout=5)
```

```
        if res.status_code != 200:
```

```
            print("Invalid country")
```

```
            return
```

```
        data = res.json()
```

```
        print("Cases:", data['cases'])
```

```
        print("Deaths:", data['deaths'])
```

```
        print("Recovered:", data['recovered'])
```

```
        print("Active:", data['active'])
```

except:

```
print("API Error")
```

```
covid_data("India")
```

```
1 # Generate Python code to fetch COVID-19 stats by country with error handling and retries.
2 import requests
3
4 def covid_data(country):
5     try:
6         url = f"https://disease.sh/v3/covid-19/countries/{country}"
7         res = requests.get(url, timeout=5)
8
9         if res.status_code != 200:
10             print("Invalid country")
11             return
12
13         data = res.json()
14
15         print("Cases:", data['cases'])
16         print("Deaths:", data['deaths'])
17         print("Recovered:", data['recovered'])
18         print("Active:", data['active'])
19
20     except:
21         print("API Error")
22
23 covid_data("India")
```

PROBLEMS OUTPUT TERMINAL PORTS

DEBUG CONSOLE

TERMINAL

```
-
Deaths: 533570
Recovered: 0
Active: 44501823
```

Unable to sy
Please open

Justification

- Uses real-time health API
- Handles invalid country inputs
- Extracts multiple data fields
- Demonstrates robust API usage

